# Intégration continue de l'API SouverainRecrut

Rapport technique

| | |
|---|---|
| **Projet** | SouverainRecrut, API Backend (Node.js 22, Express, TypeScript, Prisma, PostgreSQL) |
| **Module** | CI/CD, Mastère Expert en Développement (RNCP 40166) |
| **Dépôt** | github.com/mtbvr/CICD_souverainrecrut, branche dev |
| **Auteur** | Matéo Bouvier |
| **Date** | 9 novembre 2026 |

## Sommaire


1. Objectif et résultat ..... 1
2. Architecture ..... 2
3. Tests et couverture ..... 3
4. Analyse SonarQube ..... 3
5. Image Docker et analyse Trivy ..... 4
6. Pipeline Jenkins ..... 5


## 1. Objectif et résultat

L'objectif est d'automatiser la vérification et la publication de l'API Backend. À chaque modification de la branche dev, Jenkins compile le code, exécute les tests, lance l'analyse SonarQube, construit l'image Docker et la fait analyser par Trivy. L'image n'est publiée sur Docker Hub que si toutes ces étapes ont réussi.

Cinq builds ont été exécutés. Deux d'entre eux ont été arrêtés par un contrôle, ce qui confirme que la publication dépend bien de la réussite des vérifications.

| Build | Image | Résultat | Commentaire |
|---|---|---|---|
| 1 | 1.0.0-61c559b | Arrêté au scan Trivy | Vulnérabilité critique dans une dépendance (partie 5) |
| 2 | 1.0.0-563928b | Publié | Dépendance mise à jour |
| 3 | 1.0.0-563928b | Arrêté au Quality Gate | Quality Gate personnalisé (partie 4) |
| 4 | 1.0.0-172892d | Publié | Code corrigé |
| 5 | 1.0.0-8b039e7 | Publié | Image durcie, version de référence |




*Figure 1 : vue des étapes dans Jenkins pour les builds 1 à 4*

## 2. Architecture

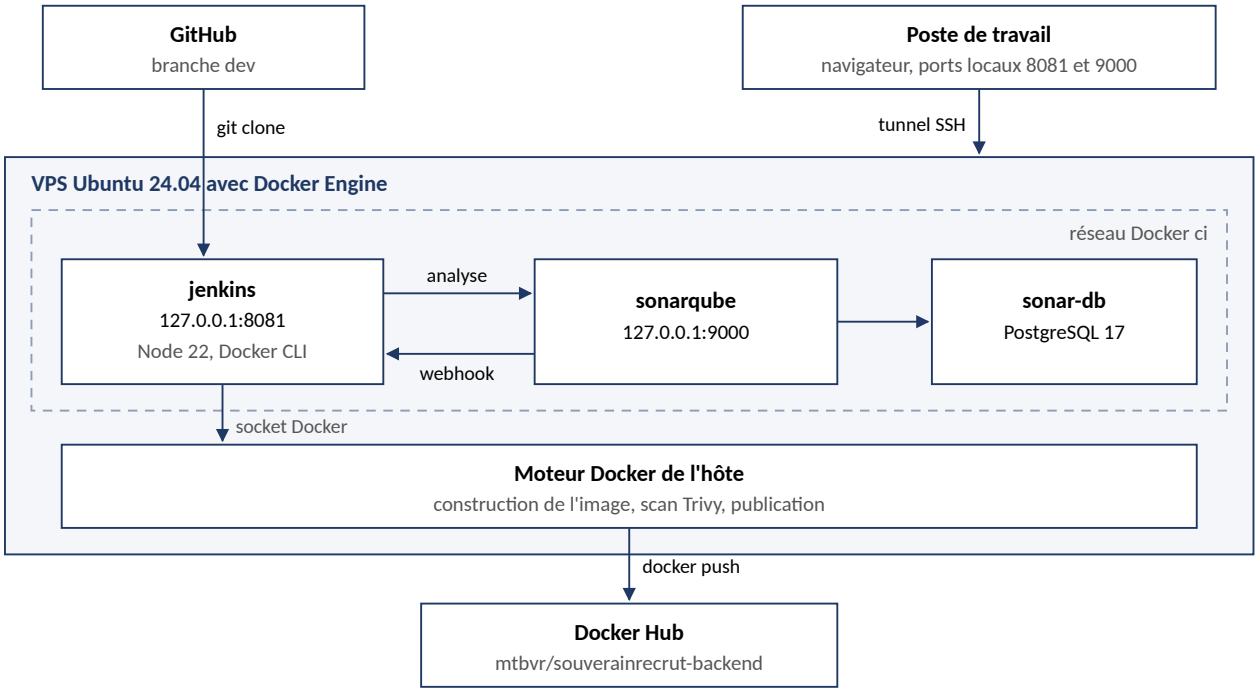



*Figure 2 : architecture de la chaîne d'intégration continue*

Jenkins, SonarQube et la base PostgreSQL de SonarQube sont déployés avec Docker Compose sur un VPS Ubuntu. Jenkins utilise le moteur Docker de l'hôte pour construire l'image et exécuter Trivy. Les interfaces web n'écoutent que sur l'adresse locale du serveur et sont consultées depuis le poste de travail par un tunnel SSH.

L'installation du lab est scriptée (setup-vm.sh et start-lab.sh). Le script check-lab.sh vérifie ensuite que les services sont opérationnels.




*Figure 3 : vérification du lab avec check-lab.sh*

## 3. Tests et couverture

Le dépôt d'origine ne contenait pas de tests. 97 tests ont été écrits avec Jest et Supertest. Ils portent sur l'authentification, la gestion des candidats, des offres, des candidatures et des utilisateurs, ainsi que sur le contrôle d'accès par rôle. Le client Prisma est simulé, ce qui permet d'exécuter les tests sans base de données.

La couverture de code atteint 100 %. Un seuil de 80 % est configuré et fait échouer le build s'il n'est pas atteint.

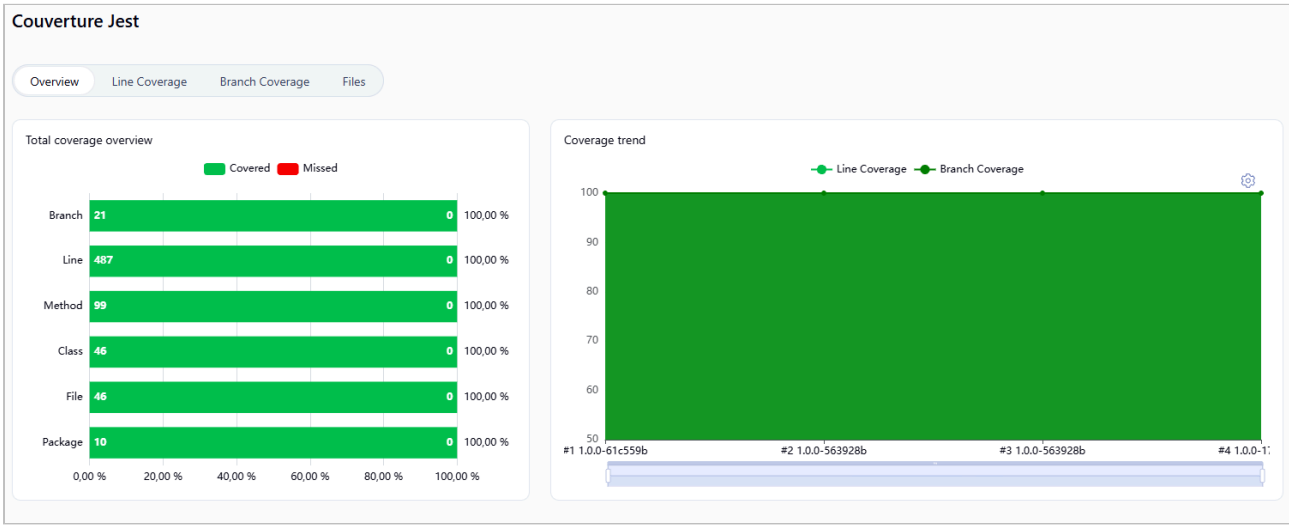



*Figure 4 : couverture de code publiée dans Jenkins*

## 4. Analyse SonarQube

Chaque build déclenche une analyse SonarQube, puis Jenkins attend le résultat du Quality Gate avant de poursuivre. Le Quality Gate par défaut (Sonar way) ne porte que sur le code nouveau. Un Quality Gate SouverainRecrut a donc été défini sur l'ensemble du code, avec les conditions suivantes : couverture d'au moins 80 %, duplication inférieure à 3 %, note A en fiabilité, en sécurité et en maintenabilité, revue de tous les Security Hotspots.

Le build 3 a échoué sur ce Quality Gate (figure 5). Deux corrections ont été apportées. L'en-tête X-Powered-By, qui révélait l'utilisation d'Express, a été désactivé, et les appels à parseInt de la pagination ont été remplacés par Number.parseInt.

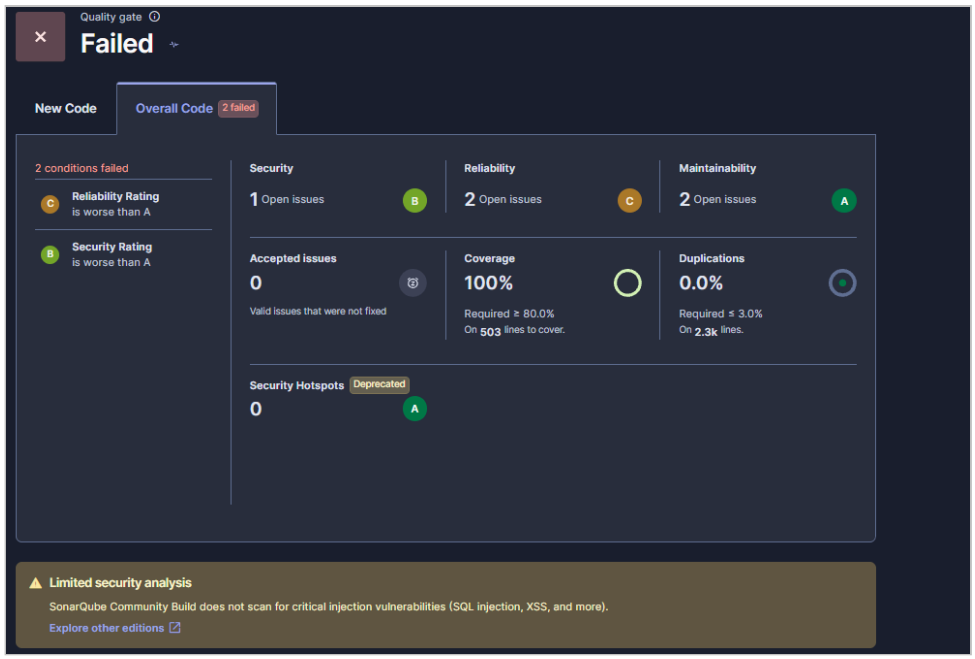



*Figure 5 : échec du Quality Gate au build 3*

Le build 4 a ensuite validé le Quality Gate, avec la note A dans toutes les catégories et aucun problème restant (figure 6).

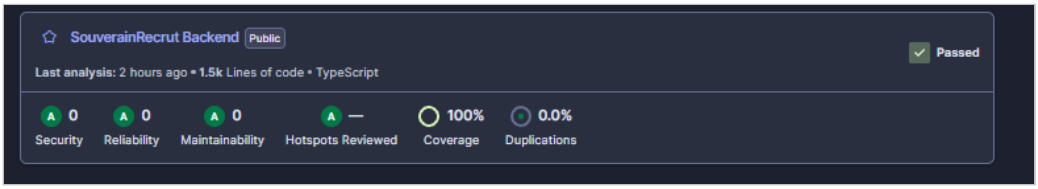



*Figure 6 : Quality Gate validé au build 4*

## 5. Image Docker et analyse Trivy

Le Dockerfile fourni a été adapté. L'image repose sur Node 22 Alpine et est construite en deux étapes, afin de n'embarquer que le code compilé et les dépendances de production. Elle s'exécute avec un utilisateur non privilégié et dispose d'un contrôle de santé. Un fichier .dockerignore exclut les fichiers inutiles, et le commit d'origine est enregistré dans les métadonnées de l'image.

Chaque image est publiée avec deux tags. Le tag version-commit, par exemple 1.0.0-8b039e7, est fixe et sert au déploiement. Le tag dev désigne la dernière image validée.

Trivy analyse l'image après sa construction. Le build est arrêté si une vulnérabilité critique dispose d'un correctif. Les autres vulnérabilités figurent dans le rapport sans bloquer la publication.

Au build 1, Trivy a détecté une vulnérabilité critique dans proxy-addr, une dépendance d'Express (figure 7). La mise à jour du paquet dans le fichier package-lock.json a corrigé le problème et le build 2 a été publié.

```
14:35:35  Node.js (node-pkg)
14:35:35  ==================
14:35:35  Total: 1 (CRITICAL: 1)
14:35:35
14:35:35  ┌──────────────────────────┬─────────────────┬──────────┬────────┬───────────────────┬───────────────┬──────────────────────────────────────────────────────────────┐
14:35:35  │         Library          │ Vulnerability   │ Severity │ Status │ Installed Version │ Fixed Version │                            Title                             │
14:35:35  ├──────────────────────────┼─────────────────┼──────────┼────────┼───────────────────┼───────────────┼──────────────────────────────────────────────────────────────┤
14:35:35  │ proxy-addr (package.json) │ CVE-2026-90711 │ CRITICAL │ fixed  │ 2.0.7             │ 2.0.8         │ proxy-addr: proxy-addr vulnerable to IP spoofing via         │
14:35:35  │                          │                 │          │        │                   │               │ IPv4-mapped IPv6 trust subnet                                │
14:35:35  │                          │                 │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-90711                   │
14:35:35  └──────────────────────────┴─────────────────┴──────────┴────────┴───────────────────┴───────────────┴──────────────────────────────────────────────────────────────┘
```

*Figure 7 : vulnérabilité critique détectée par Trivy au build 1*

Le nombre de vulnérabilités de niveau HIGH a ensuite été réduit en retirant npm de l'image finale et en mettant à jour plusieurs dépendances. L'image obtenue a été publiée par le build 5.

| Image | CRITICAL | HIGH | MEDIUM | LOW |
|---|---|---|---|---|
| 1.0.0-172892d (build 4) | 0 | 26 | 20 | 3 |
| 1.0.0-8b039e7 (build 5) | 0 | 1 | 4 | 0 |

La vulnérabilité HIGH restante concerne une dépendance de Prisma utilisée uniquement lors des migrations. Elle sera corrigée lors du passage à Prisma 7.

## 6. Pipeline Jenkins

Le pipeline est défini dans le fichier integration-continue/Jenkinsfile et s'exécute sur la branche dev. Il comprend dix étapes : récupération du code, installation des dépendances, génération du client Prisma, compilation TypeScript, tests et couverture, analyse SonarQube, Quality Gate, construction de l'image, scan Trivy et publication sur Docker Hub.

L'échec d'une étape interrompt le pipeline. La publication étant la dernière étape, une image n'est envoyée sur Docker Hub qu'après la réussite de tous les contrôles.

Le token SonarQube et le token d'accès Docker Hub sont enregistrés dans les credentials Jenkins. Le dépôt et l'image ne contiennent aucun secret.

```
15:52:26  Image publiée : <compte>/souverainrecrut-backend:1.0.0-172892d (+ tag dev)
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] archiveArtifacts
15:52:26  Archiving artifacts
[Pipeline] sh
15:52:26  + docker image ls --format {{.Repository}}:{{.Tag}}
```

*Figure 8 : publication de l'image sur Docker Hub en fin de build*